ניתוח, ניצול ומניעת חולשת React2Shell (CVE-2025-55182) באפליקציות מבוססות Next.js ו-React

מרדכי ישעיהו

מכללת עזריאלי להנדסה

תיאור החולשה והרקע התאורטי

מהי החולשה: מדובר בחולשת אבטחה קריטית מסוג Insecure Deserialization בפרוטוקול React Server Components (RSC) המוכרת גם כ-React2Shell חולשה זו מאפשרת לתוקף לבצע קוד זדוני מרחוק (Remote Code Execution - RCE) לפני שלב האימות (Pre-Authentication).

כיצד היא נוצרת: החולשה נוצרת מכיוון שהשרת מקבל אובייקטי JSON מורכבים מהלקוח (כחלק מפרוטוקול ה-Flight של React) ומבצע להם דה-סריאליזציה ללא בדיקת סוגים (Strict Type-Checking) או סניטציה מתאימה. היעדר הסינון מאפשר לתוקף להזריק מאפיינים פנימיים של JavaScript כמו __proto__ ו-constructor, מה שמוביל ל-Prototype Pollution ולשינוי שרשרת הקריאות (Callback chain) בזמן ריצה.

באיזה הקשר היא מופיעה במערכות ווב: החולשה מופיעה במערכות המשתמשות בארכיטקטורת Server Actions ו-Server Components. כאשר מופעלת בקשת POST (לרוב עם ההדר Next-Action), מנוע הריצה בצד השרת מנסה לפענח את בקשת ה-multipart/form-data כדי לנווט ולבצע את הפעולה המבוקשת בצד השרת.

מה תוקף יכול להשיג באמצעותה: ניצול מוצלח מאפשר לתוקף להריץ פקודות מערכת (OS Commands) ברמת ההרשאות של תהליך ה-Node.js. תוקף יכול להשתלט על השרת, להתקין תוכנות כופר, או לגנוב מסדי נתונים רגישים (כגון פרטי משתמשים וסיסמאות), כפי שהודגם בפרויקט.

סביבת העבודה

טכנולוגיות (שפת תכנות, סביבת עבודה מסד נתונים וכו'):

- **צד השרת והלקוח:** React 19.0.0 ו-Next.js 15.0.0 (גרסאות פגיעות).
- **שפות פיתוח:** JavaScript / TypeScript / Python.
- **סביבת ריצה:** Node.js הפועל על גבי מערכת הפעלה (Windows (Host.
- **מסד נתונים:** קובץ NoSQL (users.db) המכיל נתוני E-commerce פיקטיביים שנוצרו באמצעות ספריית Faker.

כלים המשמשים לביצוע ההדגמה:

- שפת **Python 3** עם ספריית requests להרצת סקריפט התקיפה (Weaponized Exploit).
- שורת הפקודה של (Windows (CMD / PowerShell.
- סביבת פיתוח VS Code ודפדפן לבדיקת הפעילות התקינה של האתר (Localhost / LAN IP).

מימוש והדגמת המתקפה (POC)

תיאור של ה־POC: הקמת אתר E-commerce (חנות קניות) פגיע המכיל עמוד Login ומסד נתונים פנימי. התוקף שולח סקריפט פייתון אוטומטי המייצר בקשת HTTP POST זדונית לשרת החנות, עוקף את טופס ההתחברות, ושואב את כל תוכן מסד הנתונים ישירות לטרמינל שלו ללא שום הזדהות מוקדמת.

שלבי הניצול (השלבים של המתקפה):

1. **זיהוי:** איתור אתר המריץ Next.js בגרסה פגיעה והאזנה ל-Endpoint המקבל בקשות RSC.
2. **יצירת Payload:** שימוש בסקריפט לבניית אובייקט JSON המכיל הזרקת Prototype (הוספת מפתחות then ו-constructor).
3. **הזרקת פקודת מערכת:** עטיפת פקודת Windows (למשל type data\users.db) בתוך פקודת PowerShell כדי למנוע קריסת תחביר (Syntax Error) של ה-child_process ב-Node.js.
4. **עקיפת ה-Crash (Exfiltration):** קידוד פלט הנתונים (דרך encodeURIComponent) כדי למנוע שגיאת HTTP 500 כתוצאה מתווים שבורים, והחזרת המידע הגנוב לתוקף בתוך הדר שגיאה מזויף מסוג NEXT_REDIRECT.

Payloads / קוד / בקשות לדוגמה: פקודת התקיפה כפי שהורצה בטרמינל מול כתובת ה-LAN של השרת: python react2shell-ultimate.py -u http://10.0.0.16:3000 --god --windows --cmd "cd data & type users.db"

תוצאה בפועל של הניצול: השרת מפענח את הבקשה ומריץ את הקוד. הפלט בפועל הוא הדפסה של כל תוכן קובץ ה-users.db (המכיל מזהי משתמשים, כתובות אימייל וסיסמאות) ישירות למסך הטרמינל של התוקף. לחלופין, הודגמה גם הרצת פקודת calc.exe שהקפיצה מחשבון על שרת היעד כהוכחה חותכת ל-RCE.

דיאגרמה המתארת את שלבי המתקפה:

➔ Unsafe JSON Deserialization ➔ Next.js RSC Endpoint ➔ Attacker Request
System Command ➔ Node.js child_process Exec ➔ Prototype Pollution Triggered
Response with Stolen Data ➔ (Data Theft)

מנגנוני הגנה ומניעה

מדוע ההגנות הקיימות נכשלו: מפתחי פלטפורמת React סמכו על מבנה הנתונים המגיע מהלקוח ללא אכיפה קפדנית של סכימת נתונים בשכבת הפרוטוקול. המערכת אפשרה פענוח של אובייקטי JavaScript עמוקים, מה שחשף אותה לחולשות קלאסיות של Prototype Pollution. בנוסף, חסרה הגבלת הרשאות ברמת ה-OS לביצוע קריאות מערכת דרך Node.js.

כיצד ניתן למנוע את החולשה: הפתרון המוחלט הוא עדכון התלויות בקובץ ה-package.json לגרסאות המתוקנות, בהן הוסף מנגנון Strict Type-Checking המונע קבלת מפתחות אובייקט מסוכנים (כמו __proto__).

פקודת התיקון שהורצה: react-dom@19.0.1 npm install next@15.0.5 react@19.0.1

Best Practices רלוונטיים:

- **Principle of Least Privilege:** הרצת תהליך השרת (Node.js) תחת משתמש מוגבל במערכת ההפעלה, שלא רשאי לגשת למסדי נתונים שלמים או להריץ סקריפטים של PowerShell.
- **Serialization & Input Validation:** לעולם אין לסמוך על נתוני Serialized המגיעים מהלקוח. יש להשתמש בספריות בטוחות לפענוח.
- **Web Application Firewall (WAF):** הגדרת חוקי רשת לחסימת בקשות HTTP המכילות חתימות של Payload המנסה לפנות ל-child_process או process.mainModule.

מקורות

- **מאמרים :** בלוג החוקרים של חברת Wiz בנושא חשיפת החולשה-CVE) React2Shell") 2025-55182): Critical React Vulnerability").
  - https://www.wiz.io/blog/critical-vulnerability-in-react-cve-2025-55182
- **תיעוד רשמי :** תיעוד האבטחה הרשמי של חברת Vercel / Next.js בנוגע לטיפול בחולשת CVE-2025-66478: https://nextjs.org/blog/CVE-2025-66478
  - התיעוד הרשמי במאגר הפגיעויות הלאומי של ארה"ב (NVD) עבור חולשת הבסיס CVE-2025-55182ב-React: https://nvd.nist.gov/vuln/detail/CVE-2025-55182
- **OWASP:** נכנס תחת קטגוריית A08:2021 – Software and Data Integrity Failures (כשלים ביושרת תוכנה ומידע - המכיל בתוכו.Insecure Deserialization)
  - https://owasp.org/Top10/2021/A08_2021-Software_and_Data_Integrity_Failures/
- **קישורים רלוונטיים** :הריפו הרשמי בגיטהאב של React בנוגע לתיקון הקוד (Patch) שיושם בגרסה 19.0.1 React (Pull Request #35277 - Fix security vulnerability in Server Components).
  - https://github.com/facebook/react/pull/35277